

- Access Controls
- In-built Packages
- Object Class



Access Modifiers

	class	Package	Subclass (same pkg)	Subclass (diff pkg)	World (diff pkg & <u>not subclass</u>)
public	+	+	+	+	+
protected	+	+	+	+	
<u>no modifier</u>	+	+	+		
private	+				

means → no inheritance involve

Subclass → means inheritance

→ Jb kuchhi Uchhtr likh → Int name;

protected → is used when we talk about inheritance.

Notes:

Access modifiers in Java are used to control the visibility & accessibility of classes, methods and variables.

They help enforce encapsulation by restricting access to different parts of the program.

Java provide 4 types of access modifiers to define scope & protection level :->

- Public modifier: Accessible from everywhere
- Protected modifier: Accessible within the same packages & can use subclasses to access in different packages.
- Private modifier: Accessible only within the same class
- Default modifier / No modifier: Accessible only within the same package.

⇒ When to use?

- Public: This is used for API endpoints, service classes, or utility methods shared across different parts of an application.
- Protected: Commonly used in inheritance-based design like frameworks extensions.
- Private: The idea should be use as restrictive access as possible, so private should be used as much as possible.
- Default (Package-Private): Often used in package-scoped utilities or helper classes.

Public Access Modifier

The public access modifier is specified using the keyword public. Public members are accessible from everywhere in the program. There is no restriction on the scope of public data members.

```
class MathUtils {  
    public static int add(int a, int b) {  
        return a + b;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        System.out.println(MathUtils.add(5, 10)); // accessible anywhere  
    }  
}
```

Output

15

Explanation: add() is globally accessible due to the public modifier.

Top-level classes or interfaces can not be declared as private because, private means "only visible within the enclosing class".

Protected Access Modifier

The protected access modifier is specified using the keyword protected. The methods or data members declared as protected are accessible within the same package or subclasses in different packages.

```
// File: Vehicle.java in package p1  
package p1;  
public class Vehicle {  
    protected int speed;  
}  
  
// File: Bike.java in package p2  
package p2;  
import p1.Vehicle;  
public class Bike extends Vehicle {  
    void showSpeed() {  
        speed = 100; // allowed: subclass in different package  
        System.out.println(speed);  
    }  
}  
  
// File: Test.java in package p2  
package p2;  
import p1.Vehicle;  
public class Test {  
    public static void main(String[] args) {  
        Bike b = new Bike();  
        b.showSpeed(); // prints 100  
  
        Vehicle v = new Vehicle();  
        // System.out.println(v.speed); // ERROR: cannot access protected outside package & non-subclass  
    }  
}
```

Output

Access via subclass method: 100
0

Explanation: speed is accessible via subclass methods and other classes in the same package, but direct access from a different package (non-subclass) would fail.

Private Access Modifier

The private access modifier is specified using the keyword `private`. The methods or data members declared as private are accessible only within the class in which they are declared.

```
class Person {  
    // private variable  
    private String name;  
  
    public void setName(String name) {  
        this.name = name; // accessible within class  
    }  
  
    public String getName() { return name; }  
}  
  
public class Geeks {  
    public static void main(String[] args)  
    {  
  
        Person p = new Person();  
        p.setName("Alice");  
  
        // System.out.println(p.name); // Error: 'name'  
        // has private access  
        System.out.println(p.getName());  
    }  
}
```

To access out of the class → use getter & setter.

name is only available in this class.

can't access directly as it's private

Output

Alice

Explanation: Direct access to name is not allowed outside Person, enforcing encapsulation.

Default Access Modifier

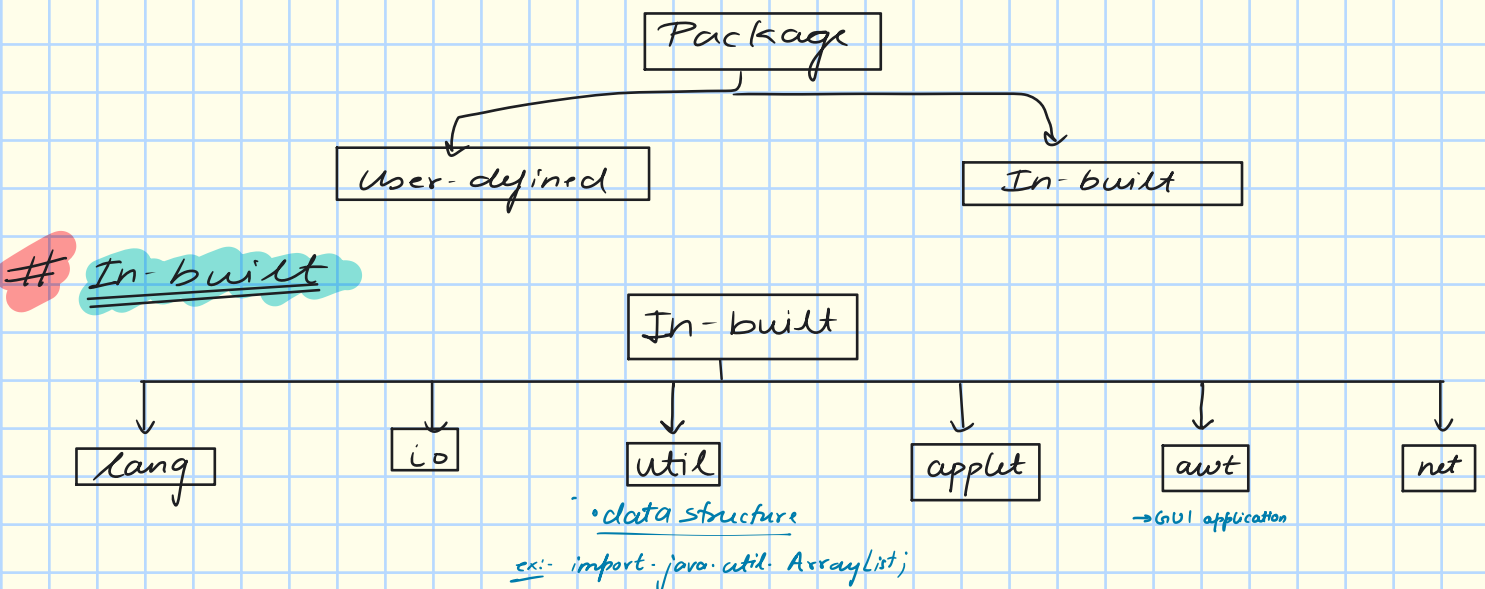
If no access modifier is specified, the member has default (package-private) access and can only be accessed within the same package. This means only classes within the same package can access it.

```
class Car {  
    String model; // default access  
}  
  
public class Main {  
  
    public static void main(String[] args){  
  
        Car c = new Car();  
        c.model = "Tesla"; // accessible within the same package  
        System.out.println(c.model);  
    }  
}
```

Output

Tesla

Explanation: Members with default access cannot be accessed from classes in a different package.



- lang :-
- Contains core classes that are fundamental to the java language.
 - It's automatically imported into every java program.

ex:- Math.max(a, b)

- IO :-
- provides classes & interfaces for handling input & output operations, including reading user input & performing file handling tasks.

- util :-
- provides utility classes such as collections, date & time utilities, and array manipulation tools.

- One of the most commonly used classes in this packages is Arrays, which offer helpful methods for array operations.

Object class

Object class (in Java.lang) is the root of the java class hierarchy. Every class in Java either directly or indirectly extends Object.

It provides essential methods like toString(), equals(), hashCode(), clone(), ... etc.

Acts as the root of all Java classes.

Defines essential methods shared by all objects.

- ⇒ HashCode :-
- method returns the hash value of an object (not its memory address).
 - Used in hash-based collections like hash map, hash set, etc.

* default hashCode give different Integer value to each object.

ex:-

```
public class ObjectDemo {
    int num;

    ObjectDemo(int num) {
        this.num = num;
    }

    @Override
    public int hashCode() {
        return super.hashCode();
    }

    public static void main(...) {
        ObjectDemo ob1 = new ObjectDemo();
        ObjectDemo ob2 = new ObjectDemo();
        System.out.println(ob1); // 41359092
        System.out.println(ob2); // 149928006

        ObjectDemo ob3 = ob2;
        System.out.println(ob3); // 149928006
    }
}
```

Have same hashCode

